Data Management for Machine Learning (DM4ML) - Assignment 2

End-to-End Data Management Pipeline for a Recommendation System

MovieLens 100K - Ingestion - Validation - Preparation - Transformation - Features - Training - Evaluation - Orchestration

Submission	Individual submission
Student	• Subham Kumar Panda (2025AG05283)
GitHub repository	https://github.com/2025ag05283/recommendation-pipeline
Pipeline run	SUCCESS - 2026-11-13 22:55:44 - 117.3 s - orchestrator: prefect
Report generated	2026-11-14 00:49

100,024 raw interactions	**34** invalid records caught	**65** documented features
3 models tracked in MLflow	**0.1572** best NDCG@10 (Hybrid)	**+17.2%** NDCG lift vs baseline

Contents

1. Overview, architecture and how to run
2. Stage 1 - Data ingestion
3. Stage 2 - Data validation
4. Stage 3 - Data preparation
5. Stage 4 - Data transformation
6. Stage 5 - Feature management
7. Stage 6 - Model training and experiment tracking
8. Stage 7 - Model evaluation
9. Stage 8 - Orchestration and reproducibility
10. Prediction using the feature store
11. Conclusions and limitations
12. Appendix - full feature catalog

1. Overview, architecture and how to run

The pipeline turns raw user, item and interaction data into evaluated recommendation models. It is implemented as independent Python modules (one per stage) that communicate only through files in dedicated storage zones (`data/raw` → `data/processed` → `data/features` → `models` → `reports`). All parameters are in `configs/config.yaml` (seed 42), and the stages are chained as a Prefect DAG.

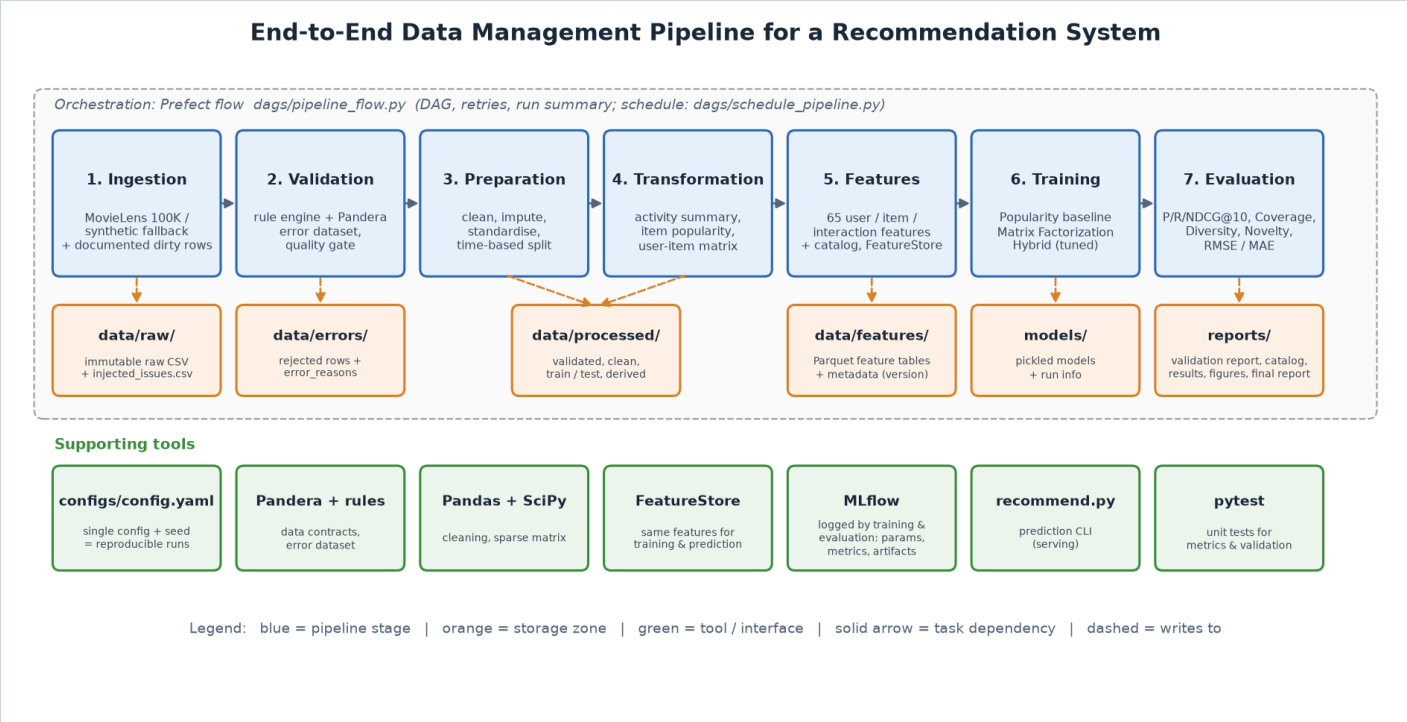


Figure 0. Pipeline architecture: stages (blue), storage zones (orange), tools (green)

Tools and design decisions

Concern	Choice	Reason
Dataset	MovieLens 100K (GroupLens)	users, items and timestamped explicit ratings, as required by the brief
Validation	Custom rule engine (20 rules) + Pandera schemas	row-level error dataset with reasons, plus a declarative data contract
Storage	Parquet (columnar, typed) + JSON metadata	fast, schema-preserving, versionable
Feature store	File-based `FeatureStore` + registry + catalog	one definition reused at training and prediction time
Models	Popularity, Matrix Factorization (SGD), Hybrid	baseline vs collaborative vs hybrid comparison
Tracking	MLflow (SQLite backend)	parameters, metrics, artifacts and lineage tags per run
Orchestration	Prefect flow with 10 tasks	explicit DAG, retries, UI evidence, cron scheduling

How to reproduce

```
python -m venv .venv  &&  .venv\Scripts\activate
python -m pip install -r requirements.txt
prefect server start                       # terminal 1
prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api
python run_pipeline.py                     # terminal 2: full DAG
mlflow ui --backend-store-uri sqlite:///mlflow.db
python recommend.py --user 42
```

2. Stage 1 - Data ingestion

Module: `src/ingestion/ingest.py`. Downloads MovieLens 100K, parses `u.user`, `u.item` and `u.data` **as text** (type checking is left to validation), and stores an immutable raw copy in `data/raw/`. To prove that validation works, a small, fully documented set of invalid records is injected (`data/raw/injected_issues.csv`).

Source	movielens
Raw users	949
Raw items	1,686
Raw interactions	100,024
Injected issues	45
Ingested at	2026-11-14 00:27:50
Duration	0.39 s

Injected data-quality issues (by type)

table	issue	records
interactions	exact duplicate row	5
interactions	missing rating	1
interactions	missing user_id	3
interactions	non-integer rating	1
interactions	rating out of range 1-5	4
interactions	timestamp before 1995	1
interactions	timestamp in the future (2100)	1
interactions	unknown item_id (referential integrity)	2
interactions	unknown user_id (referential integrity)	3
interactions	unparseable timestamp	1
interactions	wrong data type (rating)	1
interactions	wrong data type (user_id)	1
items	duplicate item_id	1
items	genre flag not 0/1	1
items	invalid release date	1
items	missing optional field 'release_date' (to be imputed)	3
items	missing title	1
users	age out of range	2
users	duplicate user_id	1
users	invalid gender code	1
users	missing age	1
users	missing optional field 'occupation' (to be imputed)	5
users	missing optional field 'zip_code' (to be imputed)	3
users	wrong data type (age)	1

Sample of raw interactions (as stored, untyped)

user_id	item_id	rating	timestamp
537	1069	2	886030938
585	1488	4	891283921
397	182	5	885349759
737	175	5	884315246
857	258	5	883432193
317	748	5	891446843

Execution evidence

```
C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>C:\Users\lenovo\project_dm4
ml\recommendation-pipeline\recommendation-pipeline\.venv\Scripts\activate.bat

(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>python run_pipeline
.py --stage ingest
2026-11-14 00:27:50 | INFO     | src.ingestion.ingest         | MovieLens 100K already present in C:\Users
\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline\data\raw\ml-100k - skipping downloa
d
2026-11-14 00:27:50 | INFO     | src.ingestion.ingest         | Loaded source files: 943 users, 1682 items
, 100000 interactions
2026-11-14 00:27:50 | INFO     | src.ingestion.ingest         | Injected 45 documented data-quality issues
 (see data/raw/injected_issues.csv)
2026-11-14 00:27:50 | INFO     | src.ingestion.ingest         | Ingestion done: {'source': 'movielens', 's
ource_folder': 'C:\\Users\\lenovo\\project_dm4ml\\recommendation-pipeline\\recommendation-pipeline\\data\
\raw\\ml-100k', 'n_users_raw': 949, 'n_items_raw': 1686, 'n_interactions_raw': 100024, 'n_injected_issues
': 45, 'seconds': 0.39, 'ingested_at': '2026-11-14 00:27:50'}

(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>
```

Figure 1. Stage 1 ingestion - ingestion

3. Stage 2 - Data validation

Modules: `src/validation/checks.py`, `schemas.py`, `validate.py`. Users and items are validated first, then interactions (so referential integrity is checked against *valid* ids). Every failing row is written to `data/errors/` with the list of rules it broke; valid rows are typed and saved to `data/processed/*_validated`. A quality gate stops the pipeline if more than 5% of interactions are invalid. Pandera available: **True**.

Row counts

table	raw rows	valid rows	rejected rows	rejected %
users	949	943	6	0.6320
items	1686	1682	4	0.2370
interactions	100024	100000	24	0.0240

Validation rules and results (18 of 20 rules detected problems)

table	check	category	rule	failed_rows	status
users	user_required_fields	missing	user_id, age and gender must be present	1	FAIL
users	user_id_integer	type	user_id must be an integer	0	PASS
users	age_integer	type	age must be an integer	1	FAIL
users	age_in_range	range	age must be within [1, 100]	2	FAIL
users	gender_allowed	range	gender must be one of ['F', 'M']	1	FAIL
users	user_id_unique	duplicate	user_id must be unique (first occurrence kept)	1	FAIL
items	item_required_fields	missing	item_id and title must be present	1	FAIL
items	item_id_integer	type	item_id must be an integer	0	PASS
items	release_date_format	timestamp	release_date, if present, must be a valid DD-Mon-YYYY date	1	FAIL
items	genre_flags_binary	type	every genre flag must be 0 or 1	1	FAIL
items	item_id_unique	duplicate	item_id must be unique (first occurrence kept)	1	FAIL
interactions	interaction_required_fields	missing	user_id, item_id, rating and timestamp must be present	4	FAIL
interactions	ids_integer	type	user_id and item_id must be integers	1	FAIL
interactions	rating_numeric	type	rating must be numeric	1	FAIL
interactions	rating_valid	range	rating must be an integer in [1, 5]	5	FAIL
interactions	timestamp_numeric	timestamp	timestamp must be a unix epoch integer	1	FAIL
interactions	timestamp_plausible	timestamp	timestamp must be between 1995-01-01 and now	2	FAIL
interactions	no_exact_duplicates	duplicate	exact duplicate interaction rows are not allowed (first kept)	10	FAIL
interactions	user_exists	referential	user_id must exist in the validated users table	4	FAIL
interactions	item_exists	referential	item_id must exist in the validated items table	2	FAIL

Sample of the error dataset (`data/errors/all_errors.csv`)

table	error_reasons	record
users	user_id_unique	2 | 53 | F | other | 94043
users	age_in_range	5001 | -3 | M | student | 12345
users	age_in_range	5002 | 250 | F | writer | 12345
users	gender_allowed	5003 | 30 | X | artist | 12345
users	user_required_fields	5004 | | M | artist | 12345
users	age_integer	5005 | thirty | M | artist | 12345
items	item_id_unique	3 | Four Rooms (1995) | 01-Jan-1995 | | http://us.imdb.com/M/title-ex
items	item_required_fields	7001 | | 01-Jan-1995 | | http://us.imdb.com/M/title-exact?Four%20Roo
items	release_date_format	7002 | Four Rooms (1995) | 31-Feb-1996 | | http://us.imdb.com/M/title
items	genre_flags_binary	7003 | Four Rooms (1995) | 01-Jan-1995 | | http://us.imdb.com/M/title
interactions	user_exists	99999 | 1 | 4 | 880000005
interactions	item_exists	1 | 88888 | 4 | 880000006

Execution evidence

```
(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>python run_pipeline
.py --stage validate
2026-11-14 00:29:06 | INFO     | src.validation.validate      | Validation: {'users': '943/949 valid', 'it
ems': '1682/1686 valid', 'interactions': '100000/100024 valid'}
2026-11-14 00:29:06 | INFO     | src.validation.validate      | Rejected 34 records -> data/errors/ ; repo
rt -> reports/validation_report.md

(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>
```

Figure 2. Stage 2 validation - validation

4. Stage 3 - Data preparation

Module: `src/preparation/prepare.py`. Standardises formats (gender upper-case, occupation lower-case, dates parsed), imputes missing optional values (occupation/zip → "unknown", missing release date → median date with an `release_date_imputed` flag), keeps only the latest rating when a user rated the same item twice, and creates a **global time-based split**: everything up to the 80% timestamp quantile is training data, later interactions are test data. This simulates production (train on the past, predict the future) and avoids look-ahead leakage.

Train/test cutoff	1998-03-07 02:21:09
Training interactions	80,003
Test interactions	19,997
Train period	1997-09-20 03:05:10 to 1998-03-07 02:21:09
Test period	1998-03-07 02:21:37 to 1998-04-22 23:10:38
Users in test	301
Cold-start test users (no training history)	192
Cold-start test items	66
Repeated ratings removed	0
Occupations imputed	5
Release dates imputed	4

Sample of the cleaned training split

user_id	item_id	rating	timestamp	datetime
259	255	4	874724710	1997-09-20 03:05:10
259	286	4	874724727	1997-09-20 03:05:27
259	298	4	874724754	1997-09-20 03:05:54
259	185	4	874724781	1997-09-20 03:06:21
259	173	4	874724843	1997-09-20 03:07:23
259	108	4	874724882	1997-09-20 03:08:02

Execution evidence

```
.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>python run_pipeline
py --stage prepare
026-11-14 00:29:45 | INFO     | src.preparation.prepare      | Preparation done. Time split at 1998-03-07
02:21:09 -> train 80003 rows, test 19997 rows (192 cold-start test users)

.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>
```

Figure 3. Stage 3 preparation - preparation

5. Stage 4 - Data transformation

Module: `src/transformation/transform.py`. Builds derived datasets from the **training split only**: a user activity summary, item popularity (with a Bayesian-average rating that shrinks items with few ratings towards the global mean), a one-hot item-genre matrix, and a sparse user-item rating matrix (943 x 1682, 80,003 ratings, density 5.04%).

Top-10 items by training popularity

popularity_rank	item_id	title	n_ratings	avg_rating	bayesian_avg_rating	user_share
1	50	Star Wars (1977)	473	4.3552	4.3378	0.5016
2	181	Return of the Jedi (1983)	423	4.0000	3.9889	0.4486
3	100	Fargo (1996)	417	4.1535	4.1386	0.4422
4	294	Liar Liar (1997)	396	3.1465	3.1556	0.4199
5	258	Contact (1997)	394	3.8173	3.8098	0.4178
6	288	Scream (1996)	383	3.4256	3.4279	0.4062
7	1	Toy Story (1995)	374	3.8797	3.8702	0.3966
8	286	English Patient, The (1996)	370	3.6865	3.6820	0.3924
9	121	Independence Day (ID4) (1996)	366	3.4372	3.4393	0.3881
10	174	Raiders of the Lost Ark (1981)	341	4.2786	4.2569	0.3616

User activity summary (distribution over 751 users)

statistic	mean	std	min	50%	max
n_interactions	106.5290	99.8270	1.0000	66.0000	737.0000
avg_rating	3.5830	0.4530	1.4920	3.6170	4.8700
rating_std	1.0130	0.2090	0.0000	1.0040	1.7500
active_span_days	20.0300	41.5120	0.0000	0.0370	167.3460

Execution evidence

```
(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>python run_pipeline
.py --stage transform
2026-11-14 00:30:22 | INFO     | src.transformation.transform | Transformation done: user-item matrix (943
, 1682), density 0.0504

(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>
```

Figure 4. Stage 4 transformation - transformation

6. Stage 5 - Feature management

Modules: `src/features/registry.py` (metadata of every feature), `build_features.py`, `store.py`. Features are computed from training data **as of the train cutoff** (1998-03-07 02:21:09), stored as Parquet tables in `data/features/` with versioned metadata (v1.0.0), and checked against the registry (built columns must equal declared columns). The same `FeatureStore` class serves features to model training *and* to prediction (`recommend.py`), which avoids training/serving skew. The full catalog is in the appendix.

Entity	Features	Rows stored
interaction	6	80,003
item	29	1,682
user	30	943

Key features (excerpt of the catalog)

name	entity	dtype	description	logic
user_activity_count	user	int	Number of training interactions	count(train rows per user)
user_avg_rating	user	float	Average rating given by the user	mean(train.rating per user)
user_days_since_last_interaction	user	float	Days between the user's last training interaction and the reference date (train cutoff)	(cutoff - max(train.datetime)) in days
user_preferred_genre	user	category	Genre the user rated most often among liked items (rating >= 4); falls back to all items	argmax over genre counts of liked items
item_popularity	item	int	Number of training ratings of the item	count(train rows per item)
item_bayesian_avg_rating	item	float	Bayesian-average rating shrunk towards the global mean	(m*mu + sum r)/(m + n), m = bayesian_prior_weight
item_age_years	item	float	Item age at the reference date	(cutoff - release_date) / 365.25
rating_dev_from_user_avg	interaction	float	Rating minus the user's average rating	rating - user_avg_rating
interaction_recency_days	interaction	float	Days between the interaction and the reference date	(cutoff - datetime) in days

Sample - user features

user_id	user_age	user_gender	user_activity_count	user_avg_rating	user_rating_std	user_preferred_genre	user_days_since_last_interaction
1	24	M	265	3.6110	1.2570	drama	5.8370
2	53	F	62	3.7100	1.0300	drama	2.9750
3	23	M	35	2.3140	0.8670	comedy	0.0000
4	24	M	0	3.5180	0.0000	unknown	-1.0000
5	33	F	175	2.8740	1.3630	comedy	116.1850

Sample - item features

item_id	item_popularity	item_avg_rating	item_bayesian_avg_rating	item_age_years	item_n_genres
1	374	3.8800	3.8700	3.1790	3
2	110	3.2180	3.2430	3.1790	3
3	80	3.0120	3.0690	3.1790	1
4	175	3.5140	3.5140	3.1790	3
5	74	3.3380	3.3590	3.1790	3

Execution evidence

```
(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>python run_pipeline
.py --stage features
2026-11-14 00:31:06 | INFO     | src.features.build_features  | Features built: 943 users x 31 cols, 1682
items x 30 cols, 80003 interactions (65 registered features)

(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>
```

Figure 5. Stage 5 feature engineering - features

7. Stage 6 - Model training and experiment tracking

Modules: `src/training/models.py`, `train.py`. All models share one interface (`fit / score / recommend / predict_rating`) and fall back to popularity for cold-start users.

- **Popularity (baseline)** - recommends the most-rated unseen items; predicts ratings with the Bayesian item average.
- **Matrix Factorization** - r(u,i) = mu + b_u + b_i + p_u.q_i, trained with SGD (train RMSE 0.5980).
- **Hybrid** - z(MF score) + w_content · z(genre affinity from the feature store) + w_pop · z(log popularity). Weights were grid-searched on the last 10% of the *training* period (the test set is never used for tuning).

Trained models (logged to MLflow with parameters, metrics, lineage tags and the model artifact)

model	MLflow run id	fit time (s)	parameters
Popularity (baseline)	3f5be1a6df11...	0.0100	ranking=training interaction count, rating_prediction=bayesian item average
Matrix Factorization (SVD)	eb18609a81c8...	30.2400	n_factors=40, n_epochs=30, learning_rate=0.01, regularization=0.05, min_item_support=5, seed=42
Hybrid (MF + content + popularity)	baf5c5e12c41...	56.8500	w_content=0.5, w_pop=2.0, base_mf_factors=40

Hybrid weight tuning - best validation configurations

w_content	w_pop	precision_at_10	recall_at_10	ndcg_at_10	hit_rate_at_10	n_users
0.5000	2.0000	0.1088	0.1073	0.1480	0.4118	68
0.5000	3.0000	0.1074	0.1039	0.1463	0.4265	68
1.0000	3.0000	0.1029	0.0987	0.1446	0.4118	68
0.2500	3.0000	0.1088	0.0853	0.1417	0.4118	68
0.2500	2.0000	0.1074	0.0997	0.1403	0.4412	68
0.0000	2.0000	0.1103	0.1049	0.1394	0.4265	68

Execution evidence

```
2026-11-14 00:31:50 | INFO     | src.training.train           | Trained popularity            in    0.0s
-> popularity.pkl
2026-11-14 00:31:50 | INFO     | src.utils.tracking           | Experiment tracking: MLflow (sqlite:///C:/
Users/lenovo/project_dm4ml/recommendation-pipeline/recommendation-pipeline/mlflow.db), experiment 'dm4ml-
recommendation-pipeline'
2026-11-14 00:31:51 | INFO     | src.training.models          |   MF epoch  1/30  train RMSE = 0.9553
2026-11-14 00:32:00 | INFO     | src.training.models          |   MF epoch 10/30  train RMSE = 0.8392
2026-11-14 00:32:10 | INFO     | src.training.models          |   MF epoch 20/30  train RMSE = 0.6925
2026-11-14 00:32:20 | INFO     | src.training.models          |   MF epoch 30/30  train RMSE = 0.5980
2026-11-14 00:32:20 | INFO     | src.training.train           | Trained matrix_factorization   in   30.2s
-> matrix_factorization.pkl
2026-11-14 00:32:20 | INFO     | src.utils.tracking           | Experiment tracking: MLflow (sqlite:///C:/
Users/lenovo/project_dm4ml/recommendation-pipeline/recommendation-pipeline/mlflow.db), experiment 'dm4ml-
recommendation-pipeline'
2026-11-14 00:32:21 | INFO     | src.training.train           | Hybrid tuning: inner-train 72002 rows, val
idation 8001 rows (cut 1998-02-17 18:36:36)
2026-11-14 00:32:21 | INFO     | src.training.models          |   MF epoch  1/30  train RMSE = 0.9578
2026-11-14 00:32:29 | INFO     | src.training.models          |   MF epoch 10/30  train RMSE = 0.8429
2026-11-14 00:32:38 | INFO     | src.training.models          |   MF epoch 20/30  train RMSE = 0.6918
2026-11-14 00:32:47 | INFO     | src.training.models          |   MF epoch 30/30  train RMSE = 0.5917
2026-11-14 00:32:47 | INFO     | src.training.train           | Hybrid tuning grid (validation NDCG@10):
 w_content  w_pop  precision_at_10  recall_at_10  ndcg_at_10  hit_rate_at_10  n_users
      0.50    2.0           0.1088        0.1073      0.1480          0.4118       68
      0.50    3.0           0.1074        0.1039      0.1463          0.4265       68
```

Figure 6. Stage 6 training - training 1 popularity and mf

```
2026-11-14 00:32:38 | INFO     | src.training.models          |   MF epoch 20/30  train RMSE = 0.6918
2026-11-14 00:32:47 | INFO     | src.training.models          |   MF epoch 30/30  train RMSE = 0.5917
2026-11-14 00:32:47 | INFO     | src.training.train           | Hybrid tuning grid (validation NDCG@10):
 w_content  w_pop  precision_at_10  recall_at_10  ndcg_at_10  hit_rate_at_10  n_users
      0.50    2.0           0.1088        0.1073      0.1480          0.4118       68
      0.50    3.0           0.1074        0.1039      0.1463          0.4265       68
      1.00    3.0           0.1029        0.0987      0.1446          0.4118       68
      0.25    3.0           0.1088        0.0853      0.1417          0.4118       68
      0.25    2.0           0.1074        0.0997      0.1403          0.4412       68
      0.00    2.0           0.1103        0.1049      0.1394          0.4265       68
      1.00    2.0           0.0971        0.1031      0.1383          0.4265       68
      0.00    3.0           0.1147        0.0745      0.1378          0.3971       68
      0.50    1.0           0.1044        0.1124      0.1344          0.4412       68
      0.00    1.0           0.1059        0.1027      0.1332          0.4118       68
      1.00    1.0           0.0956        0.1087      0.1321          0.4412       68
      0.25    1.0           0.1015        0.1067      0.1321          0.3971       68
      0.25    0.5           0.0941        0.0966      0.1190          0.3824       68
      0.00    0.5           0.0868        0.0906      0.1166          0.3824       68
      1.00    0.5           0.0868        0.0852      0.1164          0.4118       68
      0.50    0.5           0.0882        0.0757      0.1100          0.3529       68
      0.25    0.0           0.0691        0.0895      0.1032          0.3676       68
      0.50    0.0           0.0735        0.0755      0.0919          0.3676       68
      1.00    0.0           0.0676        0.0776      0.0918          0.3676       68
      0.00    0.0           0.0647        0.0777      0.0914          0.3088       68
2026-11-14 00:32:47 | INFO     | src.training.train           | Trained hybrid                in   56.9s
-> hybrid.pkl
```

Figure 7. Stage 6 training - training 2 hybrid tuning

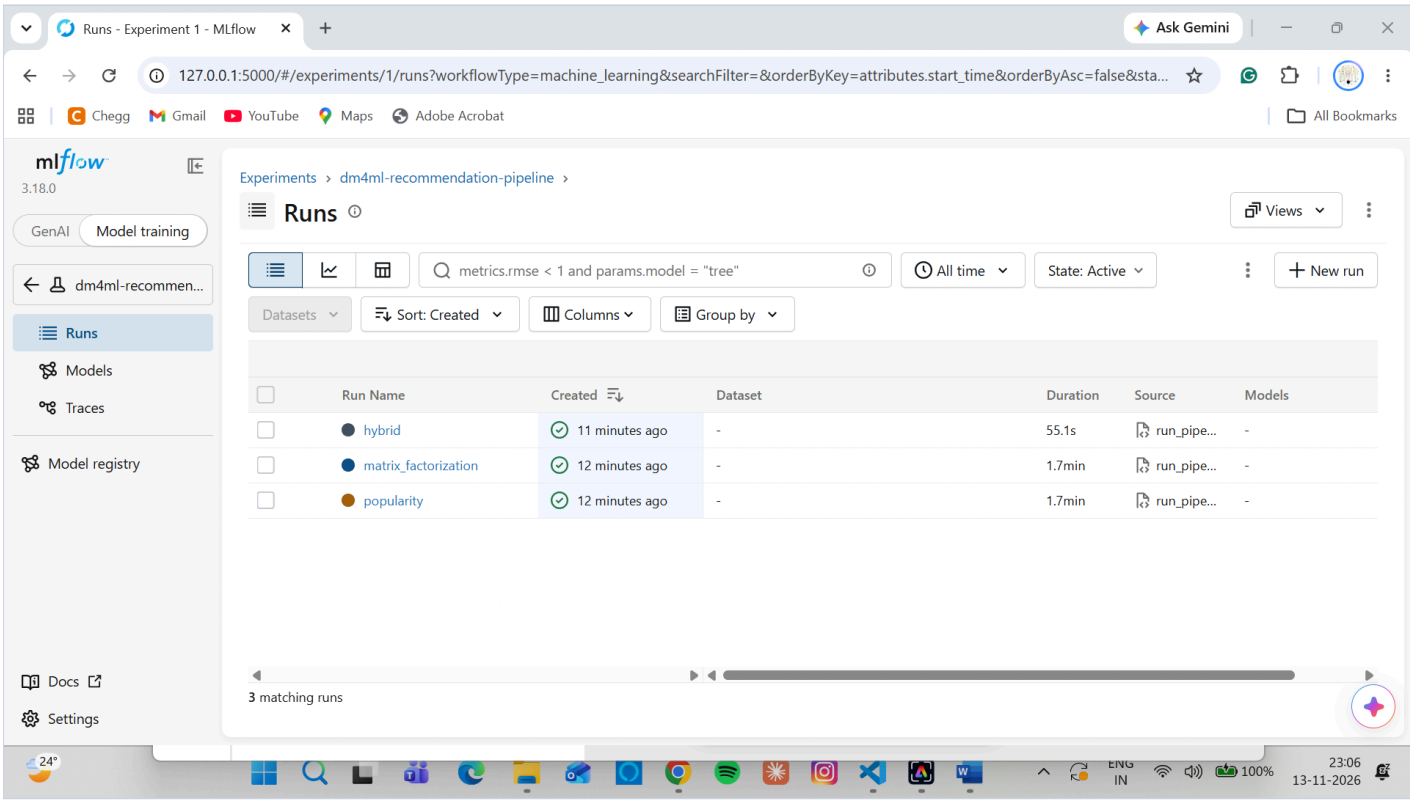


Figure 8. MLflow experiment tracking - mlflow experiment runs

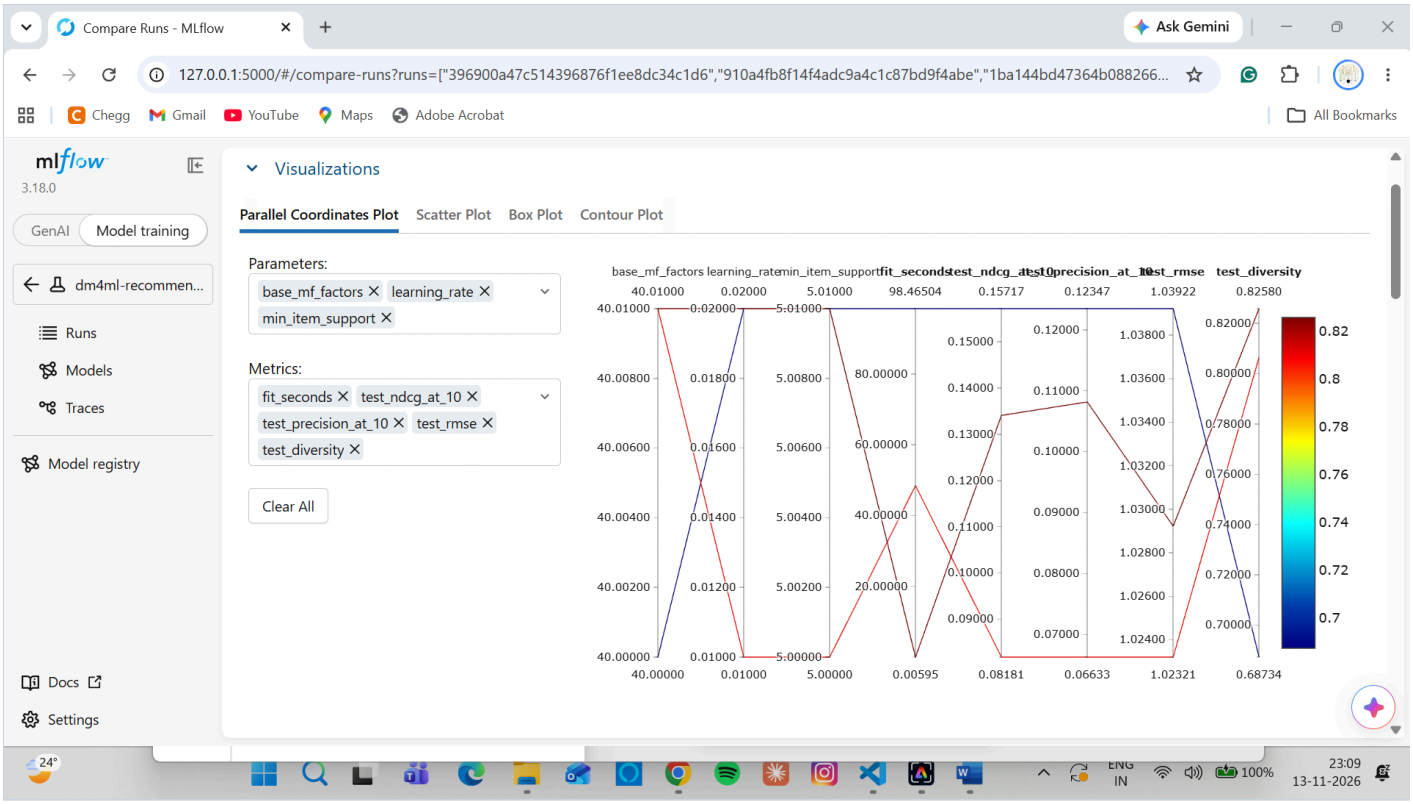


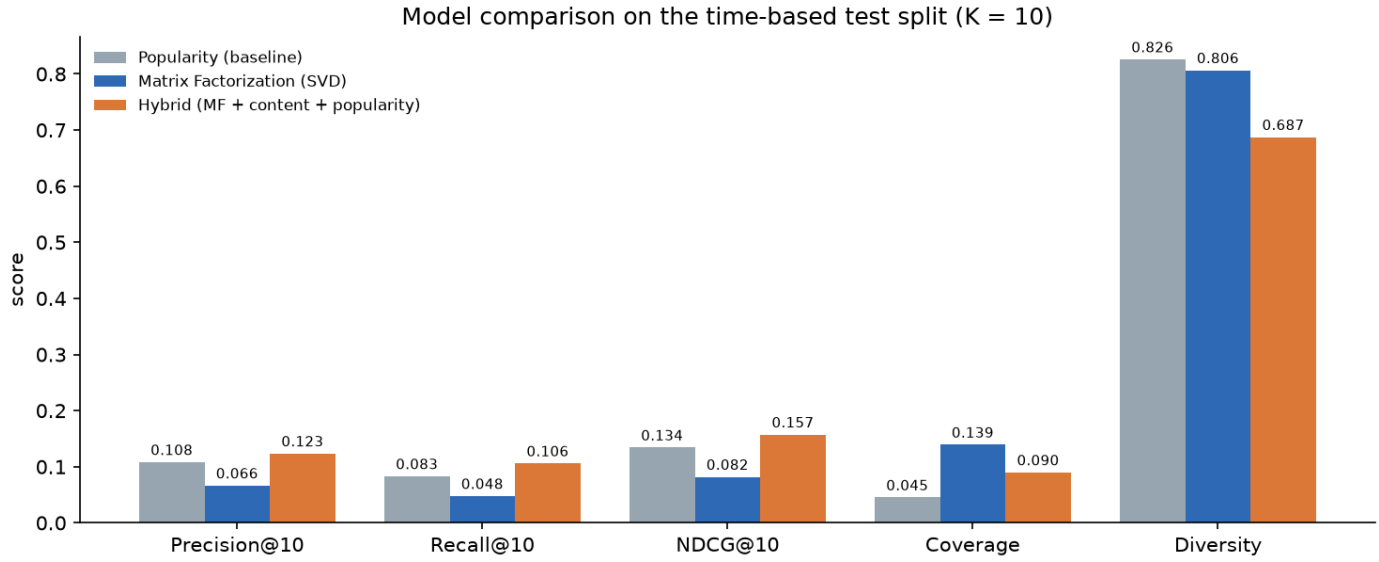
Figure 9. MLflow experiment tracking - mlflow compare runs

8. Stage 7 - Model evaluation

Modules: `src/evaluation/metrics.py` (unit-tested), `evaluate.py`. Test period = interactions after the cutoff; relevant item = test rating ≥ 4. Ranking metrics are averaged over **98 warm users** (with training history and at least one relevant test item); 192 cold-start users receive the popularity fallback. RMSE/MAE are computed on all test ratings for models that predict ratings.

Model comparison (K = 10)

model	precision_at_10	recall_at_10	ndcg_at_10	hit_rate_at_10	coverage	diversity	novelty	rmse	mae	users_evaluated
Popularity (baseline)	0.1082	0.0830	0.1341	0.4592	0.0452	0.8258	1.5823	1.0292	0.8296	98
Matrix Factorization (SVD)	0.0663	0.0476	0.0818	0.2959	0.1391	0.8064	3.3428	1.0232	0.8201	98
Hybrid (MF + content + popularity)	0.1235	0.1058	0.1572	0.4898	0.0898	0.6873	2.0929	-	-	98



Model comparison on the time-based test split

Interpretation

- Ranking quality. The best model by NDCG@10 is the hybrid model (NDCG@10 = 0.1572). The hybrid model changes NDCG@10 by +17.2% and Precision@10 by +14.2% relative to the popularity baseline.
- Matrix factorization alone ranks poorly (NDCG@10 = 0.0818) although it is the most accurate rating predictor (RMSE 1.0232 vs 1.0292 for the baseline). Minimising squared rating error does not optimise top-K ranking: MF promotes items with high predicted ratings that few users actually choose to watch. This is why the hybrid blends the MF score with popularity and genre affinity.
- Beyond accuracy. Coverage is 4.5% (popularity), 13.9% (MF) and 9.0% (hybrid); diversity is 0.826 / 0.806 / 0.687. The baseline recommends the same few blockbusters to everyone (lowest novelty), while the personalised models trade some of that safety for more varied and novel lists.
- Cold-start users (no training history) receive popularity recommendations from every model, so their NDCG@10 is identical across models (0.3153).

Example - hybrid top-10 for user 1 (1 = item was rated ≥ 4 in the test period)

rank	item_id	title	relevant_in_test
1	276	Leaving Las Vegas (1995)	0
2	285	Secrets & Lies (1996)	0
3	483	Casablanca (1942)	0
4	313	Titanic (1997)	0
5	275	Sense and Sensibility (1995)	0
6	302	L.A. Confidential (1997)	0

rank	item_id	title	relevant_in_test
7	283	Emma (1996)	0
8	318	Schindler's List (1993)	0
9	357	One Flew Over the Cuckoo's Nest (1975)	0
10	523	Cool Hand Luke (1967)	0

Execution evidence

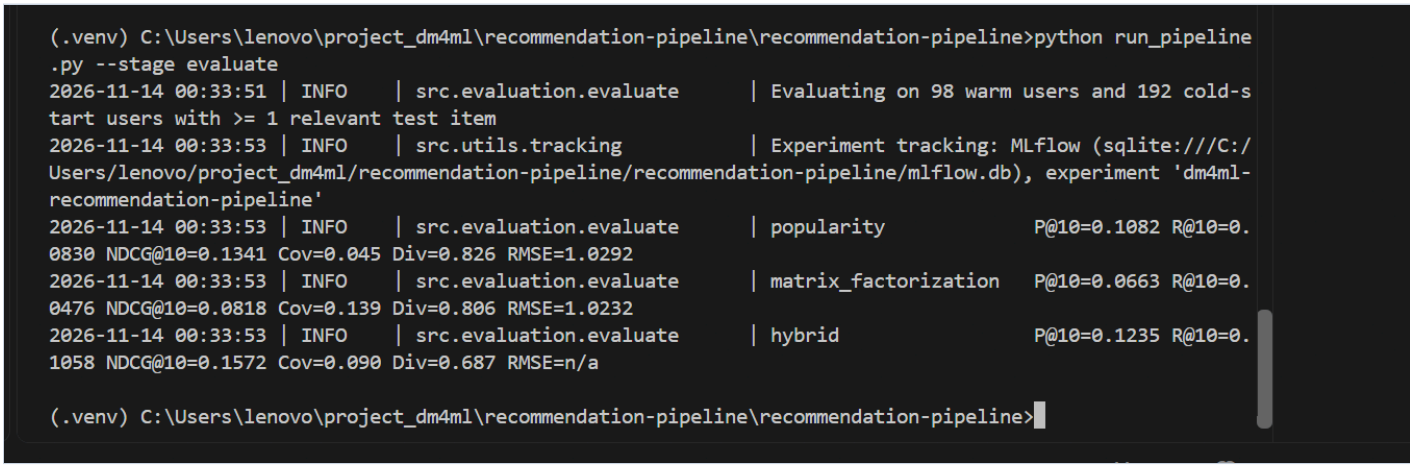


Figure 10. Stage 7 evaluation - evaluation

9. Stage 8 - Orchestration and reproducibility

Module: `dags/pipeline_flow.py`. The pipeline is a Prefect flow of 10 tasks. Each task receives the previous task's result, so Prefect draws the dependency graph: `ingest → validate → prepare → transform → build_features → train_popularity / train_matrix_factorization → train_hybrid → evaluate → generate_report`. Ingestion retries twice on failure (network), the validation quality gate can stop the run, and every run writes `reports/pipeline_run_summary.json`. `dags/schedule_pipeline.py` deploys the same flow on a daily cron schedule (02:00). One command, `python run_pipeline.py`, reproduces everything from the config.

Task run summary of the reported run (prefect, 117.3 s)

task	status	seconds
ingest	SUCCESS	1.9100
validate	SUCCESS	2.4200
prepare	SUCCESS	0.6000
transform	SUCCESS	0.4100
build_features	SUCCESS	0.3100
train_popularity	SUCCESS	6.2000
train_matrix_factorization	SUCCESS	48.9000
train_hybrid	SUCCESS	50.1500
evaluate	SUCCESS	5.3000
generate_report	SUCCESS	0.7600

Execution evidence

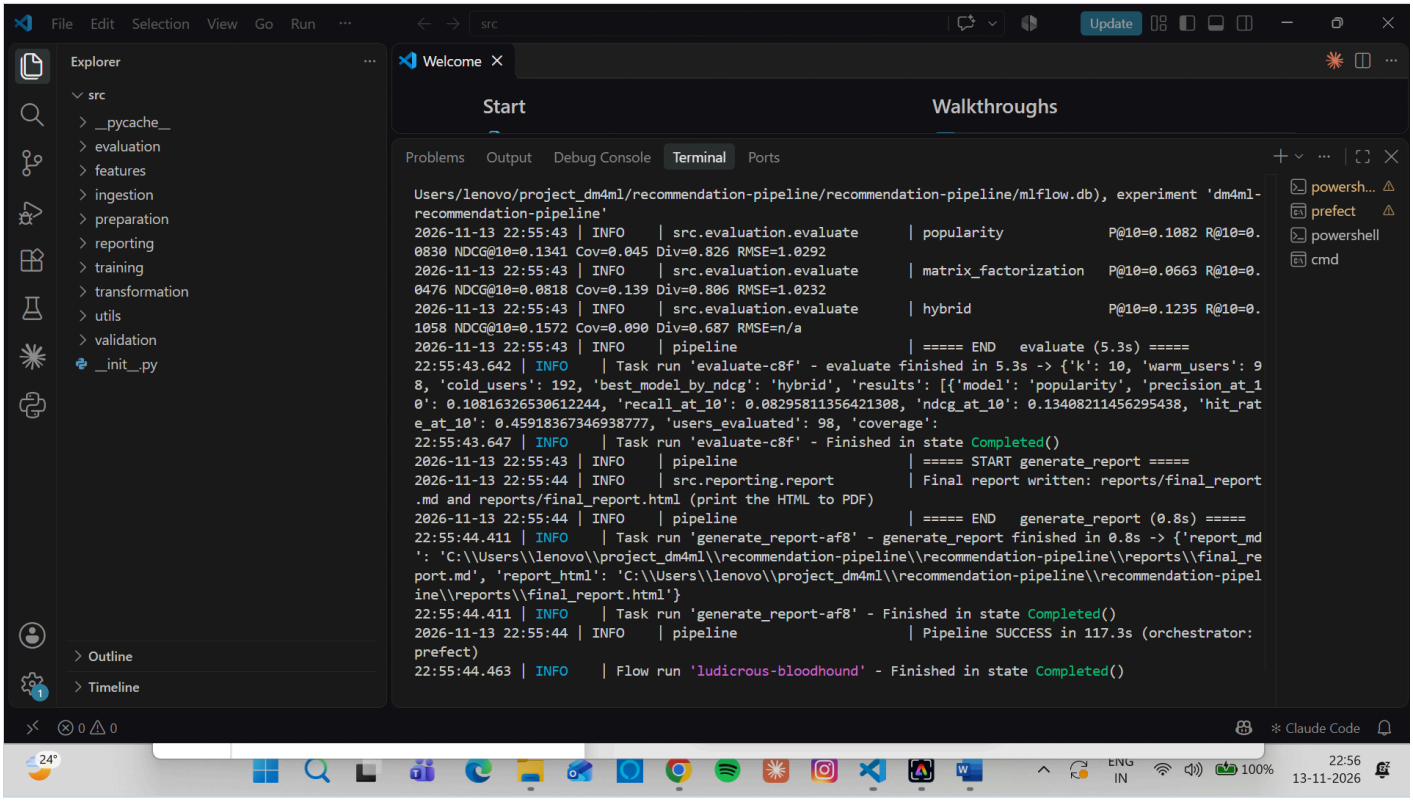


Figure 11. Full pipeline run - pipeline terminal success

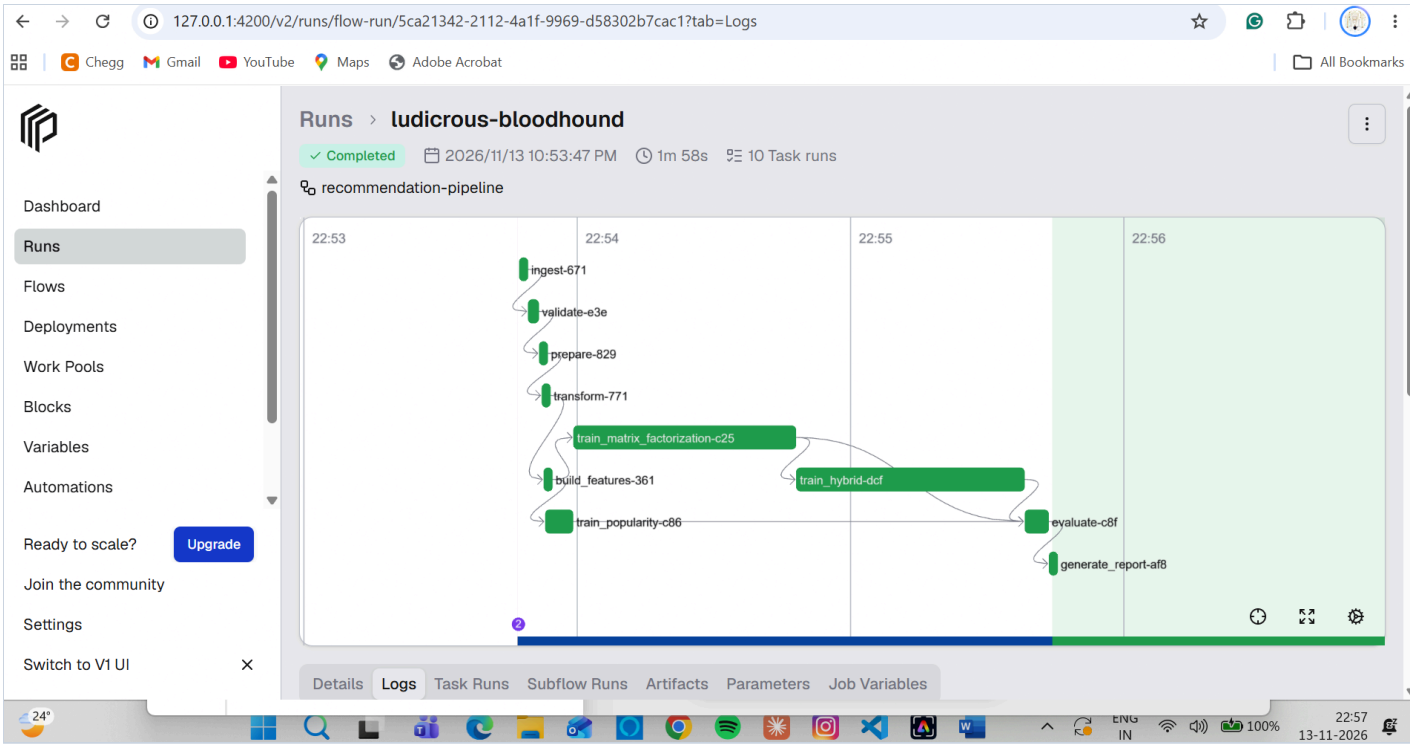


Figure 12. Full pipeline run - prefect successful run graph

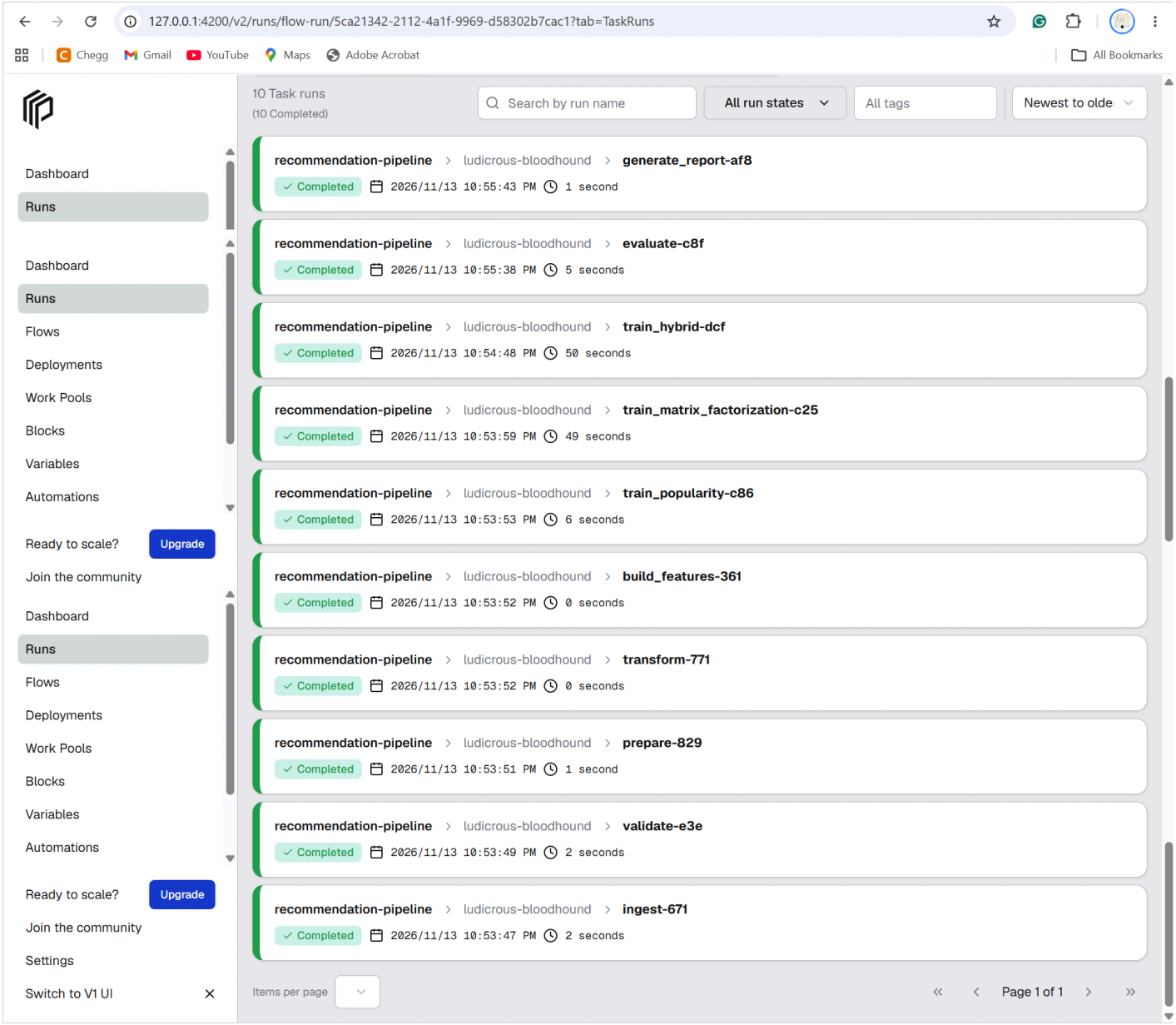


Figure 13. Full pipeline run - prefect task runs completed

```
(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>python run_pipeline
.py --stage report
2026-11-14 00:34:30 | INFO     | src.reporting.report         | Final report written: reports/final_report
.md and reports/final_report.html (print the HTML to PDF)

(.venv) C:\Users\lenovo\project_dm4ml\recommendation-pipeline\recommendation-pipeline>
```

Figure 14. Report generation - report

10. Prediction using the feature store

`recommend.py` loads a trained model and the *same* `FeatureStore` that was used in training, prints the user's features (activity, average rating, preferred genre, recency) and the top-K recommendations with item features.

```
python recommend.py --user 42
```

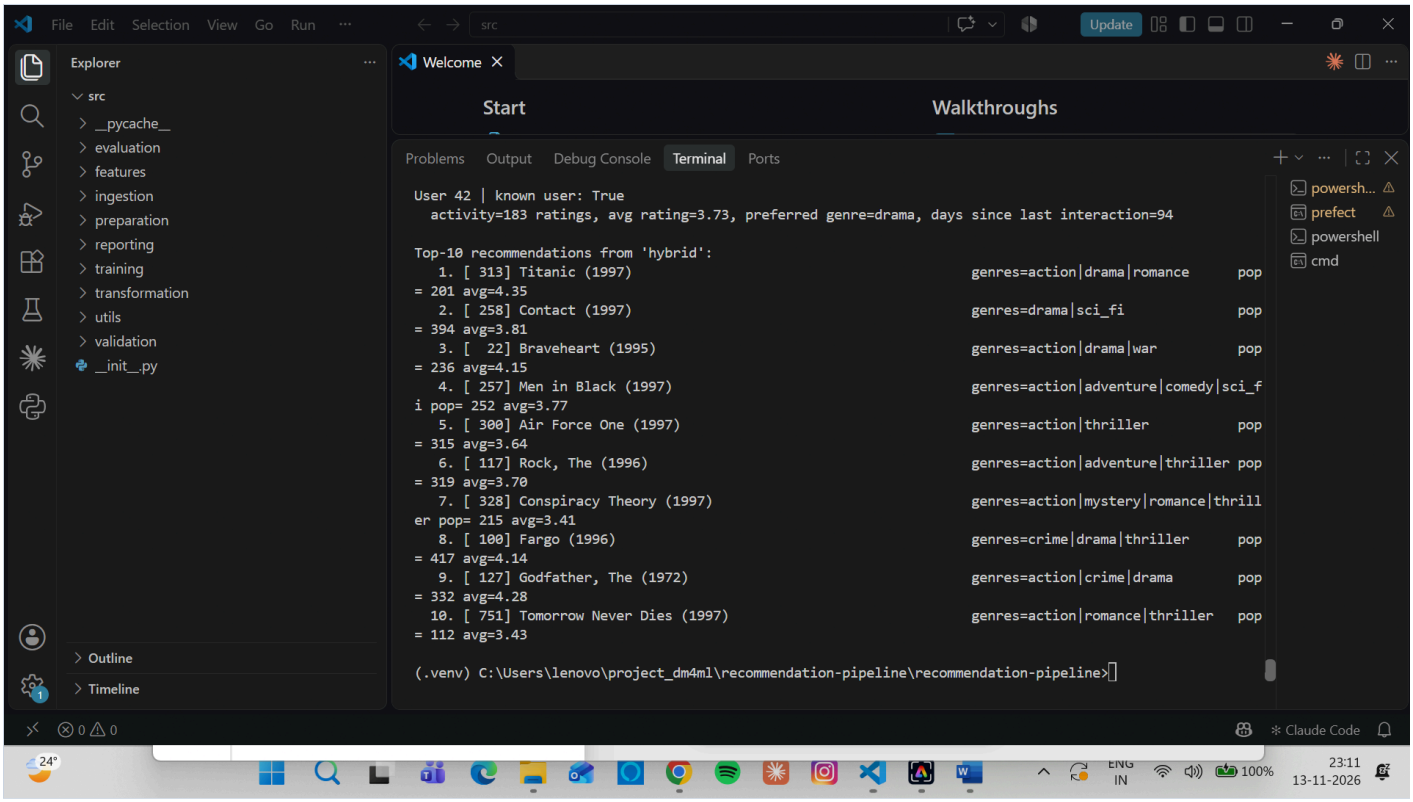


Figure 15. Prediction demo - recommend cli demo

11. Conclusions and limitations

- All eight stages run end-to-end as an orchestrated, reproducible DAG; every stage writes inspectable outputs and summaries.
- Validation caught every injected problem (34 rejected records) and documented each failure with its rule.
- The best model by NDCG@10 is **Hybrid (MF + content + popularity)** (0.1572, +17.2% vs the popularity baseline).
- **Limitations:** one global time split with offline metrics only (a rolling backtest or online A/B test would be more reliable); 192 test users are cold-start and get popularity recommendations; MovieLens 100K is small and dense; MF optimises rating error rather than ranking.
- **Future work:** ranking objectives (BPR / learning-to-rank), demographic cold-start models, Feast or a database-backed feature store, and multi-seed / multi-window evaluation.

Appendix - full feature catalog (65 features)

name	entity	dtype	description	logic	used_by	null_pct
user_age	user	int	Age of the user	users.age (validated)	profile, hybrid	0.0000
user_gender	user	category	Gender code (M/F)	users.gender upper-cased	profile	0.0000
user_occupation	user	category	Occupation (missing -> 'unknown')	users.occupation cleaned	profile	0.0000
user_age_group	user	category	Age bucket	pd.cut(age, [0,17,24,34,44,54,120])	profile	0.0000
user_activity_count	user	int	Number of training interactions	count(train rows per user)	hybrid, analysis	0.0000
user_avg_rating	user	float	Average rating given by the user	mean(train.rating per user)	hybrid, MF diagnostics	0.0000
user_rating_std	user	float	Std. dev. of the user's ratings (0 if one rating)	std(train.rating per user)	analysis	0.0000
user_days_since_last_interaction	user	float	Days between the user's last training interaction and the reference date (train cutoff)	(cutoff - max(train.datetime)) in days	recency, analysis	0.0000
user_active_span_days	user	float	Days between first and last training interaction	max(datetime) - min(datetime)	analysis	0.0000
user_preferred_genre	user	category	Genre the user rated most often among liked items (rating >= 4); falls back to all items	argmax over genre counts of liked items	explanations, hybrid	0.0000
user_n_distinct_genres	user	int	Number of distinct genres the user interacted with	count(genres with > 0 interactions)	diversity analysis	0.0000
user_genre_pref_unknown	user	float	Share of the user's liked items that belong to genre 'unknown'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_action	user	float	Share of the user's liked items that belong to genre 'action'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_adventure	user	float	Share of the user's liked items that belong to genre 'adventure'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_animation	user	float	Share of the user's liked items that belong to genre 'animation'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_childrens	user	float	Share of the user's liked items that belong to genre 'childrens'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_comedy	user	float	Share of the user's liked items that belong to genre 'comedy'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_crime	user	float	Share of the user's liked items that belong to genre 'crime'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_documentary	user	float	Share of the user's liked items that belong to genre 'documentary'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_drama	user	float	Share of the user's liked items that belong to genre 'drama'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_fantasy	user	float	Share of the user's liked items that belong to genre 'fantasy'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_film_noir	user	float	Share of the user's liked items that belong to genre 'film_noir'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_horror	user	float	Share of the user's liked items that belong to genre 'horror'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_musical	user	float	Share of the user's liked items that belong to genre 'musical'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_mystery	user	float	Share of the user's liked items that belong to genre 'mystery'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_romance	user	float	Share of the user's liked items that belong to genre 'romance'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_sci_fi	user	float	Share of the user's liked items that belong to genre 'sci_fi'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_thriller	user	float	Share of the user's liked items that belong to genre 'thriller'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_war	user	float	Share of the user's liked items that belong to genre 'war'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
user_genre_pref_western	user	float	Share of the user's liked items that belong to genre 'western'	sum(liked items with genre) / sum(all genre flags of liked items)	hybrid (content affinity)	0.0000
item_popularity	item	int	Number of training ratings of the item	count(train rows per item)	popularity, hybrid	0.0000
item_popularity_rank	item	int	Rank by popularity (1 = most popular)	rank(item_popularity, desc)	analysis	0.0000
item_user_share	item	float	Fraction of all users who rated the item	item_popularity / n_users	novelty	0.0000

name	entity	dtype	description	logic	used_by	null_pct
item_avg_rating	item	float	Mean training rating of the item (NaN-safe: global mean if unrated)	mean(train.rating per item)	analysis	0.0000
item_bayesian_avg_rating	item	float	Bayesian-average rating shrunk towards the global mean	(m*mu + sum r)/(m + n), m = bayesian_prior_weight	popularity rating prediction	0.0000
item_rating_std	item	float	Std. dev. of the item's ratings	std(train.rating per item)	analysis	0.0000
item_release_year	item	int	Release year (median-imputed when missing)	year(release_date)	content	0.0000
item_age_years	item	float	Item age at the reference date	(cutoff - release_date) / 365.25	content	0.0000
item_days_since_last_interaction	item	float	Days since the item was last rated (NaN-safe: large value if never)	(cutoff - max(train.datetime per item)) in days	trend analysis	0.0000
item_n_genres	item	int	Number of genres of the item	sum(genre flags)	content	0.0000
item_genre_unknown	item	int	1 if the item belongs to genre 'unknown'	u.item genre flag 'unknown'	hybrid (content), diversity	0.0000
item_genre_action	item	int	1 if the item belongs to genre 'action'	u.item genre flag 'action'	hybrid (content), diversity	0.0000
item_genre_adventure	item	int	1 if the item belongs to genre 'adventure'	u.item genre flag 'adventure'	hybrid (content), diversity	0.0000
item_genre_animation	item	int	1 if the item belongs to genre 'animation'	u.item genre flag 'animation'	hybrid (content), diversity	0.0000
item_genre_childrens	item	int	1 if the item belongs to genre 'childrens'	u.item genre flag 'childrens'	hybrid (content), diversity	0.0000
item_genre_comedy	item	int	1 if the item belongs to genre 'comedy'	u.item genre flag 'comedy'	hybrid (content), diversity	0.0000
item_genre_crime	item	int	1 if the item belongs to genre 'crime'	u.item genre flag 'crime'	hybrid (content), diversity	0.0000
item_genre_documentary	item	int	1 if the item belongs to genre 'documentary'	u.item genre flag 'documentary'	hybrid (content), diversity	0.0000
item_genre_drama	item	int	1 if the item belongs to genre 'drama'	u.item genre flag 'drama'	hybrid (content), diversity	0.0000
item_genre_fantasy	item	int	1 if the item belongs to genre 'fantasy'	u.item genre flag 'fantasy'	hybrid (content), diversity	0.0000
item_genre_film_noir	item	int	1 if the item belongs to genre 'film_noir'	u.item genre flag 'film_noir'	hybrid (content), diversity	0.0000
item_genre_horror	item	int	1 if the item belongs to genre 'horror'	u.item genre flag 'horror'	hybrid (content), diversity	0.0000
item_genre_musical	item	int	1 if the item belongs to genre 'musical'	u.item genre flag 'musical'	hybrid (content), diversity	0.0000
item_genre_mystery	item	int	1 if the item belongs to genre 'mystery'	u.item genre flag 'mystery'	hybrid (content), diversity	0.0000
item_genre_romance	item	int	1 if the item belongs to genre 'romance'	u.item genre flag 'romance'	hybrid (content), diversity	0.0000
item_genre_sci_fi	item	int	1 if the item belongs to genre 'sci_fi'	u.item genre flag 'sci_fi'	hybrid (content), diversity	0.0000
item_genre_thriller	item	int	1 if the item belongs to genre 'thriller'	u.item genre flag 'thriller'	hybrid (content), diversity	0.0000
item_genre_war	item	int	1 if the item belongs to genre 'war'	u.item genre flag 'war'	hybrid (content), diversity	0.0000
item_genre_western	item	int	1 if the item belongs to genre 'western'	u.item genre flag 'western'	hybrid (content), diversity	0.0000
rating	interaction	int	Explicit rating (label)	train.rating	MF training target	0.0000
is_relevant	interaction	int	1 if rating >= relevance_threshold	rating >= 4	ranking labels	0.0000
rating_dev_from_user_avg	interaction	float	Rating minus the user's average rating	rating - user_avg_rating	analysis, bias modelling	0.0000
rating_dev_from_item_avg	interaction	float	Rating minus the item's average rating	rating - item_avg_rating	analysis, bias modelling	0.0000
interaction_recency_days	interaction	float	Days between the interaction and the reference date	(cutoff - datetime) in days	recency weighting	0.0000
user_interaction_seq	interaction	int	Order of the interaction in the user's history (1 = first)	cumcount over user, sorted by time	sequence analysis	0.0000